You said:

Guide step by step logistic regression in supervised learning

ChatGPT said:

Logistic Regression in Supervised Learning: Step-by-Step Guide

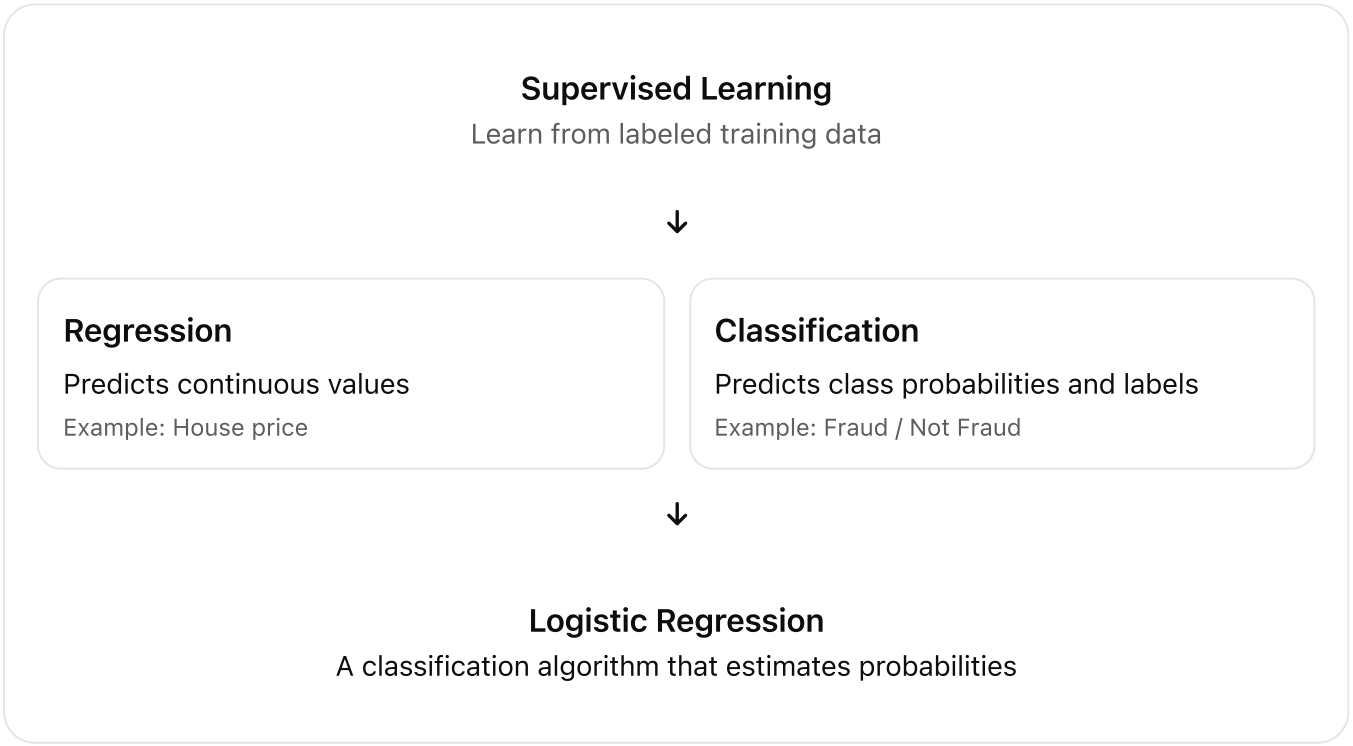
Logistic Regression is a **supervised machine learning algorithm used for classification**. It predicts the probability that an observation belongs to a particular class.

For example, you can use Logistic Regression to predict:

- Whether a customer will purchase a product: Yes/No
- Whether a bank customer will default on a loan: Yes/No
- Whether an email is spam: Spam/Not Spam
- Whether a chemical compound has high or low popularity
- Whether a customer belongs to a High, Medium, or Low risk category (multiclass classification)

Let's learn it from the fundamentals to practical Python implementation, model evaluation, and interview preparation.

Step 1: Understand where Logistic Regression fits



Despite the word *regression* in its name, Logistic Regression is primarily used for classification.

Logistic Regression types

Type	Classes	Example
Binary	2	Fraud / Not Fraud
Multinomial	3 or more unordered classes	Product category
Ordinal	Ordered categories, using suitable ordinal models	Low / Medium / High risk

Standard Logistic Regression implementations such as scikit-learn's `LogisticRegression` support binary and multiclass classification.

Step 2: Understand the mathematical concept

Logistic Regression calculates a score from the input features and converts that score into a probability between 0 and 1.

For a single input feature x, its probability is:

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Where:

- $P(Y = 1 \mid x)$: probability of belonging to class 1.
- x: input feature.
- β_0: intercept.
- β_1: coefficient of the feature.
- e: mathematical constant, approximately 2.718.

For multiple features, the model combines their weighted values before applying the same logistic function.

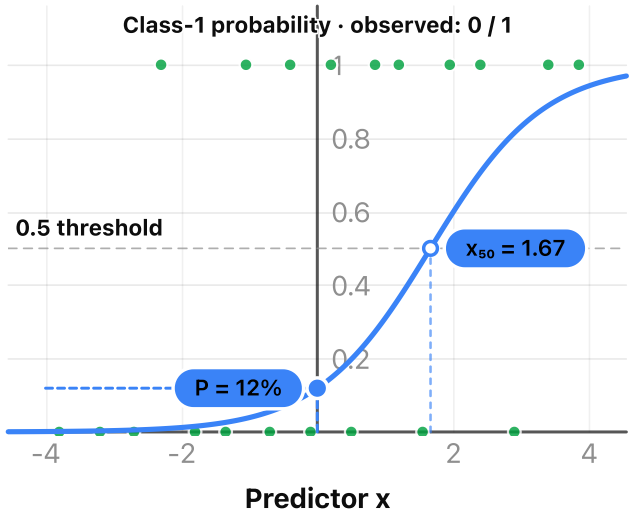
$$P(Y = 1 \mid x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x)}}$$

$$z = -2, \quad P(Y = 1 \mid x) = 12\%$$

Below the 0.5 threshold: class 0

x 0.0

β_1 1.2



Give feedback

How does the model make a prediction?

Suppose a bank wants to predict whether a customer will default on a loan.

- Input: customer's debt-to-income ratio.
- Output: probability of loan default.
- If the probability is at least 0.5, classify the customer as a defaulter.
- Otherwise, classify the customer as a non-defaulter.

The threshold of 0.5 is a common starting point, not a universal rule. Banks may choose a different threshold based on business costs and risk policies.

For example:

Predicted probability	Threshold	Predicted class
0.20	0.50	No default
0.45	0.50	No default
0.72	0.50	Default
0.91	0.50	Default

The model learns its coefficients during training by minimizing **log loss**, also called binary cross-entropy in the binary case.

Step 3: Prepare your Python environment

We'll use Python, Pandas, Matplotlib, and scikit-learn.

Install the required libraries:

```
pip install pandas numpy matplotlib scikit-learn
```

Import the libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)
from sklearn.pipeline import make_pipeline
```

Step 4: Create a sample dataset

We'll use a simple loan-default example so you can understand the full workflow before applying it to your own dataset.

```
data = {
    "income": [
        25000, 30000, 35000, 40000, 45000,
        50000, 55000, 60000, 65000, 70000,
        28000, 32000, 42000, 52000, 62000,
        27000, 38000, 48000, 58000, 68000
    ],
    "debt_ratio": [
        0.85, 0.80, 0.75, 0.70, 0.65,
        0.40, 0.35, 0.30, 0.25, 0.20,
        0.90, 0.82, 0.68, 0.45, 0.28,
        0.88, 0.72, 0.50, 0.32, 0.22
    ],
    "default": [
        1, 1, 1, 1, 1,
        0, 0, 0, 0, 0,
```

```
        1, 1, 1, 0, 0,
        1, 1, 0, 0, 0
    ]
}

df = pd.DataFrame(data)

print(df.head())
print(df.shape)
print(df["default"].value_counts())
```

Here:

- `income` and `debt_ratio` are input features.
- `default` is the target variable.
- `1` means default; `0` means no default.

This is an illustrative dataset, not real banking data.

Step 5: Perform exploratory data analysis (EDA)

Before training, inspect the dataset.

```
print(df.info())
print(df.isnull().sum())
print(df.describe())
print(df["default"].value_counts())
```

Check the following:

1. Are there missing values?
2. Are numeric columns stored as numeric types?
3. Are there duplicate records?
4. Is one class much more common than the other?
5. Are the features relevant to the target?

For a real project, investigate outliers and data leakage as well. Do not use information that would only become available after the prediction is made.

Step 6: Separate features and target

```
X = df[["income", "debt_ratio"]]
y = df["default"]
```

- `X` contains the independent variables.
- `y` contains the dependent variable or label.

This separation tells the model which information it can use to make predictions and what it must learn to predict.

Step 7: Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

Why split the data?

- **Training set:** Used to learn model parameters.
- **Testing set:** Used to evaluate performance on unseen records.
- `test_size=0.25` : Allocates 25% of records to testing.
- `random_state=42` : Makes the split reproducible.
- `stratify=y` : Attempts to preserve the class proportions in both sets.

For a very small dataset like this example, metrics can vary substantially with the split. Real projects generally need more observations and, ideally, a separate validation strategy.

Step 8: Scale features and train Logistic Regression

Scaling can help the optimization process, especially when features have different numerical ranges. We'll use a pipeline so the scaler learns its parameters from the training data only.

```
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, random_state=42)
)

model.fit(X_train, y_train)

print("Model training completed!")
```

What happens internally?

1. `StandardScaler()` standardizes the input features.
2. `LogisticRegression()` learns coefficients and an intercept.
3. The optimization process minimizes the training loss, subject to the model's regularization settings.
4. The trained model can now estimate probabilities and predict classes.

Step 9: Make predictions

Once the model is trained, predict the class labels and probabilities.

```
y_pred = model.predict(X_test)

y_prob = model.predict_proba(X_test)[:, 1]

results = X_test.copy()
results["Actual"] = y_test
results["Predicted"] = y_pred
results["Default_Probability"] = y_prob

print(results)
```

Example output format:

Income	Debt ratio	Actual	Predicted	Default probability
30,000	0.80	1	1	0.86
60,000	0.30	0	0	0.14

Illustrative values only. Actual results depend on the training/test split and fitted model.

`predict_proba()` returns estimated probabilities for each class. In this example, `[:, 1]` selects the probability of class 1, meaning default.

Step 10: Evaluate the model

A classification model should be evaluated using more than accuracy alone.

```
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

print("ROC-AUC:", roc_auc_score(
    y_test,
    model.predict_proba(X_test)[:, 1]
))
```

Understand the evaluation metrics

Accuracy

Percentage of all predictions that are correct.

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Of all customers predicted to default, how many actually defaulted?

$$\frac{TP}{TP + FP}$$

Recall

Of all actual defaulters, how many did the model identify?

$$\frac{TP}{TP + FN}$$

⚖ **F1-score**

Harmonic mean of precision and recall. Useful when you need to balance both.

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}$$

- **Confusion matrix:** Shows true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).
- **ROC-AUC:** Measures how well the model ranks positive examples above negative examples across different thresholds. A score of 0.5 indicates chance-level ranking; 1.0 indicates perfect ranking on the evaluated data.

For loan-default prediction, false negatives can be especially costly because actual defaulters may go undetected. The best metric and decision threshold depend on the bank's business requirements.

Step 11: Predict for a new customer

Suppose a new customer has an income of ₹45,000 and a debt ratio of 0.65.

```
new_customer = pd.DataFrame({
    "income": [45000],
    "debt_ratio": [0.65]
})

prediction = model.predict(new_customer)[0]
probability = model.predict_proba(new_customer)[0, 1]

print("Predicted class:", prediction)
print("Default probability:", round(probability, 4))
```

Interpret the result:

- Predicted class `0` : predicted non-default.
- Predicted class `1` : predicted default.
- Probability: estimated likelihood of the default class, according to the trained model.

The probability is not automatically a reliable real-world risk estimate; calibration and external validation may be needed before using it for actual lending decisions.

Step 12: Understand the coefficients

You can inspect the model's learned coefficients.

```
lr = model.named_steps["logisticregression"]
scaler = model.named_steps["standardscaler"]

print("Intercept:", lr.intercept_)
print("Coefficients:", lr.coef_)

print("Features:", X.columns.tolist())
```

Because we scaled the features, these coefficients correspond to standardized inputs. A positive coefficient increases the log-odds of class 1 as that feature increases, holding other features constant; a negative coefficient decreases the log-odds.

Remember that a coefficient describes an association learned by the model, not necessarily a causal relationship.

Step 13: How to use Logistic Regression with your own dataset

For a dataset with columns such as `pool_row`, `inchikey14`, `pubchem_cid`, `monoisotopic_mass`, `log1p_substances`, `log1p_pubmed`, `log1p_patents`, and `pop`, first decide what outcome you want to predict.

For example, if `pop` represents a numeric popularity score and you have defined classes such as High, Medium, and Low, you could build a multiclass Logistic Regression model.

A basic workflow would be:

1. Define the target classes from `pop` using meaningful domain-specific thresholds or an existing trusted label.
2. Exclude identifiers that do not provide useful predictive information.
3. Convert numeric features to numeric types and handle missing values.
4. Encode suitable categorical features.
5. Split the data into training and testing sets.
6. Scale numeric features and fit a multiclass Logistic Regression model.
7. Evaluate per-class precision, recall, F1-score, and the confusion matrix.
8. Check for data leakage and validate the results on unseen data.

Do not assume that a model is performing well simply because accuracy is high. In particular, if a popularity class is directly derived from `pop`, do not use `pop` itself as an input feature when predicting that class.

Step 14: Common interview questions

Question	Key answer
Is Logistic Regression supervised?	Yes, it learns from labeled examples.
Is it regression or classification?	Primarily a classification algorithm.
Why use the sigmoid function?	To map a score to a probability between 0 and 1.
What loss function is used?	Log loss for binary classification.
What is the default classification threshold?	Commonly 0.5 in binary classification.
Does Logistic Regression require feature scaling?	Not strictly, but scaling is often helpful.
Can it handle multiple classes?	Yes, using multinomial or one-vs-rest approaches.
What is regularization?	A penalty that helps control model complexity and reduce overfitting.
What is the difference from Linear Regression?	Linear Regression predicts continuous values; Logistic Regression estimates class probabilities.
What is a limitation?	It may perform poorly when the relationship between features and class log-odds is not adequately represented by the model.

Your next learning step

Since Logistic Regression is a classification algorithm, the natural next step is to compare it with Decision Trees, Random Forests, and KNN using the same dataset and evaluation metrics.

You can also implement this workflow in PySpark if you want to practice distributed machine learning on larger datasets.